

Tervezd meg a GitHub adaptert

- Project: sportmate-interview
- Storage: active
- Session ID: 01a0fbd1-3dac-7712-ab98-c51615989b80
- Started: 2026-10-02T08:53:09.254Z
- Updated: 2026-10-02T09:24:06.144Z
- Model: gpt-6-astra

This PDF is a classified, derived reading view. Raw JSONL remains the canonical lossless representation when included.

User – 2026-10-02T08:53:12.838Z

A feladatod készíteni egy szerver oldali interface-t, amiben async ready-only/get methodokat definiálunk.

A célja az, hogy bármilyen remote git providerre lehessen implementációt belőle, például Github vagy GitLab.

A GitLab implementációt nem kell elkészítened, csak is kizárólag a GitHub implementációt.

class / interface:

- közös GitSource:
 - getAccountType(string: name) -> AccountType: user | organization
 - getName() -> String
 - getRepositories() -> List<RemoteRepository>

Közös, az interface-ben lévő methodok:

- getRepositories(source: GitSource) -> List<RemoteRepository>
- getName() -> azt implementáció neve, pl "GitHub" vagy "GitLab"#{x20;

A github specifikus dolgokat a github implementációjába rakd. Tehát ha például szükség lenne egy repository valamilyen metaadatára pl a stars-ra, de az nem feltétlen van minden git provideren, akkor értelemszerűen ne a közös interface/abstract class-ba kerüljön.#{x20;

Validáld az elképzelésemet, mondd el a véleményedet a tervemről. Fontos, hogy bővíthető, könnyen átlátható, és typesafe-nek kell lennie. Nézz át mindent, olvasd be a releváns skilleket, olvasd át a kódot és tervezz.#{x20;

Az implementációt még ne kezd el. Tegyé fel nekem kérdéseket, és ha minden kérdésedet megválaszoltam akkor rakd össze a tervet. Ha a tervet le okéztam, akkor kezdheted csak az implementációt.

Assistant question – 2026-10-02T08:55:45.172Z

Ris, mit jelentsen az async működés? Például a `laravel` és a `vuejs` repository-lekérése indulhasson el egymás bevárása nélkül, vagy csak a böngésző maradjon használható a szerver válaszáig?

- Valódi async eredményt kérek. (Recommended): A PHP-metódus típusos várható eredményt ad vissza; például két source lekérését elindíthatod, mielőtt bármelyik eredményét megvárod.
- Csak a felület legyen aszinkron.: A PHP-metódus közvetlenül repository-listát ad vissza; például a későbbi Vue-kérés alatt a szerver bevárja a GitHubot.
- Háttérfeladatot szeretnék.: A lekérés queue jobként fut; például a kliens feladatazonosítót kap, és később kérdezi le az elkészült repository-listát.

Ris, milyen szerepet szánasz a `GitSource`-nak? Például a `laravel` source saját maga is tudjon repository-kat lekérni, vagy csak adatokat adjon át a GitHub providernek?

- A source delegáljon. (Recommended): A `source.getRepositories()` a hozzárendelt providerhez delegál; például a GitHub provider deríti fel a `laravel` fióktípusát, a source pedig tárolja azt.
- A source csak adatot tároljon.: A `GitSource` változtathatatlan adatobjektum lesz; például a repository-kat kizárólag a `provider.getRepositories(source)` hívás kéri le.
- A source legyen önálló adapter.: Minden source-hoz provider-specifikus objektum készül; például a `GitHubSource('laravel')` közvetlenül intézi a GitHub-lekéréseket.

Ris, meddig tartson az első implementáció? Jelenleg az oldalsáv mock source-okat mutat; például a `laravel` kiválasztása még nem küld szerveroldali kérést.

- Csak szerveroldali réteget kérek. (Recommended): Elkészülnek a PHP-contractok, típusok, a GitHub provider és tesztjeik; például a meglévő oldalsáv továbbra is mock adatokkal működik.
- GET végpontokat is kérek.: A PHP-réteg mellé lekérdezhető HTTP-végpontok is készülnek; például kérhető lesz a `laravel` repository-listája, de a Vue-oldalt még nem kötjük rá.
- A felületet is kössük rá.: A meglévő source-kezelést élő adatokhoz kötjük; például a `laravel` kiválasztásakor betöltjük és megjelenítjük a repository-kat.

User answer – 2026-10-02T08:57:55.358Z

delivery_scope: Csak szerveroldali réteget kérek. (Recommended)

source_responsibility: A source delegáljon. (Recommended)

async_semantics: Arra gondoltam az async alatt, hogy például valamilyen api-t használva (pl github api) ott http kérésekkel kérdezzük le ezeket az adatokat, és mivel mindegyik gitprovider az valószínűleg egy api-t használ, így ezeknek a methodoknak async-oknak kell lenniük. Én gondoltam túl, vagy másra gondolsz?

Assistant question – 2026-10-02T08:58:47.113Z

Ris, a különbség alapján megfelelő a szinkron PHP-interface? Például a `source.getRepositories()` HTTP-n lekéri a GitHub adatait, és kész, típusos listával tér vissza.

- Igen, szinkron interface legyen. (Recommended): A `getRepositories()` kész repository-listát ad vissza; például a hívó rögtön bejárhatja a `laravel` repository-jait.

- Mégis valódi async interface kell.: A `getRepositories()` Promise-t ad vissza; például a `laravel` és a `vuejs` lekérését együtt indíthatja a hívó, majd később várja be őket.

Ris, milyen repository-kat kell látnia az első GitHub implementációnak? Például elég a `laravel` nyilvános listája, vagy egy tokennel elérhető privát céges repository-nak is meg kell jelennie?

- Csak publikus repository-kat kérek. (Recommended): A `laravel` nyilvános repository-jait kérjük le; opcionális szerveroldali token használható, de a lista akkor is kizárólag publikus.

- Privát repository-k is kellenek.: A szerveren beállított token hozzáféréseit is használjuk; például a token tulajdonosának privát repository-jai és elérhető szervezeti repository-k is bekerülnek.

Ris, a `getRepositories()` teljes listát adjon vissza, vagy kezeljen a hívó külön oldalakat? Például egy 250 repository-val rendelkező szervezetnél ez több GitHub-kérést jelent.

- Teljes listát kérek. (Recommended): A provider belül végiglapozza a GitHub válaszait; például mind a 250 repository egyetlen típusos listában érkezik vissza.

- Oldalanként kérem az eredményt.: A közös interface lapozási paramétert és oldaleredményt kap; például először 100 repository-t kérsz, majd külön a következő oldalt.

User answer – 2026-10-02T09:00:54.046Z

pagination_behavior: Teljes listát kérek. (Recommended)

repository_visibility: a `.env` fájlomba már létrehoztam egy github personal access token-t, így ha annak van jogosultsága olvasni privát repository-kat, akkor azokra is menjen. Szerintem az api kérést ez nem érinti. Minden kérést a PAT-al küldünk, aztán vagy vissza adja a kért adatot, vagy nem. Publikus esetén értelemszerűen mindig vissza adja

confirm_sync: Igen, szinkron interface legyen. (Recommended)

Assistant question – 2026-10-02T09:02:34.943Z

Ris, egy source repository-listájába csak az adott fiók tulajdonában lévő repository-k tartozzanak? Például a személyes fiókod source-a mutassa-e a céges szervezet repository-jait is, amelyekhez a PAT-od hozzáfér?

- Csak a source sajátjai. (Recommended): A személyes source csak a saját tulajdonú repository-kat adja vissza; például a céges repository-k a cég külön source-a alatt jelennek meg.

- Minden elérhető repository kell.: A token tulajdonosának személyes source-a az együttműködőként elérhető repository-kat is listázza; például a céges repository ugyanott is

megjelenhet.

Ris, milyen adatot adjon a közös `RemoteRepository` az első verzióban? Például egy `laravel/framework` találatból csak megjeleníthető név és link kell, vagy klónozási és láthatósági adatok is?

- Kis közös adatmodellt kérek. (Recommended): A mezők: providerhez kötött azonosító, név, teljes név, webes URL és opcionális leírás; például `framework`, `laravel/framework` és a GitHub-oldal linkje.

- Bővebb közös adatmodellt kérek.: Az alapmezők mellé HTTPS/SSH klónozási URL, alapértelmezett branch és láthatóság is kerül; például a hívó közvetlenül megkapja a klónozás adatait.

Ris, a stars csak az elkülönítés példája volt, vagy már most legyen lekérhető GitHub-specifikus adat? Például a GitHub repository-objektumból elérhető legyen a csillagok száma a közös contract bővítése nélkül?

- Egyelőre csak példa volt. (Recommended): Most csak a közös repository-adatokat képezzük le; például a `stars` mező egy későbbi GitHub-specifikus bővítésben jelenik meg.

- A stars már most kell.: A GitHub repository külön, típusos `stars` mezőt kap; például a GitHubot ismerő hívó kiolvashatja, a közös contract viszont nem írja elő.

- Több GitHub-metaadat is kell.: A `stars` mellett a forks száma, az elsődleges nyelv és az archivált állapot is bekerül a GitHub-típusba; például GitHub-specifikus rendezéshez rendelkezésre állnak.

User answer – 2026-10-02T09:04:55.781Z

github_metadata_scope: Több GitHub-metaadat is kell.

repository_ownership: Szerintem ez a privát/nem privát kérdést engedjük el: csak publikus repository-ra kell, hogy működjön. Ez szerintem sok mindent megkönnyít, és jelenleg overkill lenne integrálni a privát repository-k működését. Ettől függetlenül a github api elvárja a PAT-ot a kérésekhez, de ennek járj utána. Szóval maradunk csak a publikus repository-knál.

common_repository_fields: Kis közös adatmodellt kérek. (Recommended)

Assistant question – 2026-10-02T09:06:01.609Z

Ris, mi történjen, ha nincs beállítva a `GITHUB\_PAT`? A GitHub publikus adatokhoz token nélkül is használható; például egy új fejlesztő így külön token létrehozása nélkül is lekérhetné a `laravel` repository-jait.

- Működjön PAT nélkül is. (Recommended): Ha van `GITHUB\_PAT`, minden kérésben elküldjük; ha nincs, például a `laravel` publikus listáját hitelesítés nélkül kérjük le.

- A PAT legyen kötelező.: Token hiányában konfigurációs hibát adunk; például az új fejlesztő a `laravel` lekérése előtt beállítja a saját `GITHUB\_PAT` értékét.

User answer – 2026-10-02T09:06:57.520Z

missing_pat_policy: Amennyiben nem szükséges PAT egyáltalán a publikus adatok lekéréséhez, így egyáltalán nem kell PAT, teljesen kiszedhető. Nézz utána ennek a github dokumentációjában.

Final report – 2026-10-02T09:10:44.379Z

<proposed_plan>

Ris, ****közös Git-provider interface és GitHub implementáció****

Összefoglaló

Az alapelképzelés jó: kis közös szerződés, elkülönített providerlogika, típusos eredmények. A pontosítás: a provider intézi a távoli felderítést; a `GitSource` egy konkrét fiókot képvisel, és delegálja a repository-lekérést.

Szinkron PHP-metódusok készülnek HTTP-alapú GitHub implementációval. Kizárólag szerveroldali réteg; frontend-bekötés, HTTP-végpont, adatbázis és GitLab implementáció nélkül.

****PAT nem szükséges:**** a fiókfelderítés és mindkét repository-listázó endpoint támogatja a publikus adatok hitelesítés nélküli lekérését. [Fiók API \(https://docs.github.com/en/rest/users/users#get-a-user\)](https://docs.github.com/en/rest/users/users#get-a-user), [repository API \(https://docs.github.com/en/rest/repos/repos\)](https://docs.github.com/en/rest/repos/repos)

Közös szerződés és típusok

Az új réteg az `App\Git` névtérbe kerül, elkülönített GitHub implementációval.

```
interface GitProvider
{
    public function getName(): string;

    public function getAccountType(string $name): AccountType;

    public function getSource(string $name): GitSource;

    /** @return list<RemoteRepository> */
    public function getRepositories(GitSource $source): array;
}
```

- ****`AccountType`:**** string-backed enum, `user` és `organization` értékekkel.
- ****`GitSource`:**** változtathatatlan objektum, amely tárolja a providert, a felderített fióknevet és fióktípust. `getName()` például `laravel`; `getAccountType()` paraméter nélküli, helyi getter. `getRepositories()` a providernek delegál.
- ****`getSource(name)`:**** felderíti a fiókot és létrehozza a source-ot. Konstruktorban nincs hálózati kérés. A source az őt létrehozó providerpéldányhoz kötött; más providernek átadva argumentumhibát okoz.
- ****`getAccountType(name)`:**** a provider távoli lekérdezése. A már létrehozott source saját getteréhez nem kell ismételt HTTP-kérés.

A közös `RemoteRepository` readonly adatobjektum:

Mező	Típus
<code>`id`</code>	<code>`string`</code> , providerrel együtt egyedi, például <code>`github:123`</code>
<code>`name`</code>	<code>`string`</code> , például <code>`framework`</code>
<code>`fullName`</code>	<code>`string`</code> , például <code>`laravel/framework`</code>
<code>`url`</code>	<code>`string`</code> , webes repository-URL
<code>`description`</code>	<code>?string`</code>

A readonly `GitHubRepository` ezt bővíti: ``stars: int``, ``forks: int``, ``language: ?string``, ``archived: bool``.

A közös interface `list<RemoteRepository>` típust ígér; a konkrét GitHub-metódus pontosabb `list<GitHubRepository>` típust dokumentál. A GitHub-mezők így csak megfelelően típusozott GitHub-objektumon érhetők el. Natív PHP-típusok, pontos PHPDoc és Larastan ellenőrzés; általános `mixed`` metadata-zsák nélkül.

GitHub működés

- Laravel HTTP client és dependency injection; a `GitProvider`` alapértelmezett implementációja `GitHubProvider``. `getName()`` értéke `GitHub``.
- Fiókfelderítés: `GET /users/{name}``, a válasz `login`` és `type`` mezőjéből. A GitHub névellenőrzése és normalizálása kizárólag ebben az implementációban él.
- User repository-k: `/users/{name}/repos``, `type=owner``. Organization repository-k: `/orgs/{name}/repos``, `type=public``. Csak a source saját publikus repository-jai szerepelnek, forkokkal és archivált elemekkel együtt.
- Teljes lista: oldalanként legfeljebb 100 elem, `full_name`` szerinti növekvő rendezés, a `Link`` fejléc következő oldalának követése. Azonosító szerinti deduplikálás; hibás vagy ismétlődő lapozási link esetén hiba, csonka sikeres eredmény helyett.
- Fix `https://api.github.com`` cél, GitHub JSON Accept fejléc, User-Agent és rögzített `2026-03-10`` API-verzió. Kapcsolódási timeout 3 másodperc, kérés timeout 10 másodperc.
- Minden kérés hitelesítés nélkül fut. A helyi `.env`` felesleges `GITHUB_PAT`` bejegyzése kikerül; tokenkonfiguráció nem készül.
- A külső JSON mezőit ellenőrizzük az objektumképzés előtt. Hibás mezőtípus vagy ismeretlen accounttípus válaszhibát okoz.
- Közös `GitProviderException`` hibacsalád, külön felismerhető source-not-found és rate-limit kivételekkel. Hálózati vagy API-hiba nem válhat üres listává. V1-ben nincs automatikus újrapróbálás vagy cache.

Token nélkül az alaperet ****60 kérés/óra, IP-címenként****; minden lapozási kérés fogyasztja. Rate limit esetén a kivétel elérhetővé teszi az újrapróbálás időpontját, ha a GitHub megadja. [GitHub rate limits \(https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api\)](https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api)

Ellenőrzés és elfogadás

Pest tesztek, `Http::fake()` és `Http::preventStrayRequests()` használatával:

- User és organization felismerése; source-getterek, delegálás és provider-egyezés.
- Közös mezők és GitHub-metaadatok helyes leképezése, nullable mezőkkel együtt.
- Üres lista, többoldalas teljes lista, duplikációk és hibás lapozás.
- Kizárólag GET-kérések, Authorization fejléc nélkül; megfelelő endpointok és publikus repository-szűrés.
- Nem létező fiók, rate limit, tiltott hozzáférés, serverhiba, kapcsolatmegszakadás és hibás válasz.
- Egyszerű testprovider igazolja, hogy a közös source GitHub-függőség nélkül működik.

Elfogadás: célzott tesztek, Pint, Larastan és teljes meglévő tesztcsomag sikeres. A README rövid használati példát és a token nélküli működés korlátját kapja. Új csomag nem szükséges.

Implementáció csak a terv jóváhagyása után.

</proposed_plan>

User – 2026-10-02T09:11:37.071Z

PLEASE IMPLEMENT THIS PLAN:

Ris, ****közös Git-provider interface és GitHub implementáció****

Összefoglaló

Az alapelképzelés jó: kis közös szerződés, elkülönített providerlogika, típusos eredmények. A pontosítás: a provider intézi a távoli felderítést; a `GitSource` egy konkrét fiókot képvisel, és delegálja a repository-lekérést.

Szinkron PHP-metódusok készülnek HTTP-alapú GitHub implementációval. Kizárólag szerveroldali réteg; frontend-bekötés, HTTP-végpont, adatbázis és GitLab implementáció nélkül.

****PAT nem szükséges:**** a fiókfelderítés és mindkét repository-listázó endpoint támogatja a publikus adatok hitelesítés nélküli lekérését. [Fiók API \(https://docs.github.com/en/rest/users/users#get-a-user\)](https://docs.github.com/en/rest/users/users#get-a-user), [repository API \(https://docs.github.com/en/rest/repos/repos\)](https://docs.github.com/en/rest/repos/repos)

Közös szerződés és típusok

Az új réteg az `App\Git` névtérbe kerül, elkülönített GitHub implementációval.

```
interface GitProvider
{
    public function getName(): string;

    public function getAccountType(string $name): AccountType;

    public function getSource(string $name): GitSource;

    /** @return list<RemoteRepository> */
    public function getRepositories(GitSource $source): array;
}
```

- **AccountType**: string-backed enum, `user` és `organization` értékekkel.
- **GitSource**: változtathatatlan objektum, amely tárolja a providert, a felderített fióknevet és fióktípust. `getName()` például `laravel`; `getAccountType()` paraméter nélküli, helyi getter. `getRepositories()` a providernek delegál.
- `getSource(name)`: felderíti a fiókot és létrehozza a source-ot. Konstruktorban nincs hálózati kérés. A source az őt létrehozó providerpéldányhoz kötött; más providernek átadva argumentumhibát okoz.
- `getAccountType(name)`: a provider távoli lekérdezése. A már létrehozott source saját getteréhez nem kell ismételt HTTP-kérés.

A közös `RemoteRepository` readonly adatobjektum:

Mező	Típus
<code>id</code>	<code>string</code> , providerrel együtt egyedi, például <code>github:123</code>
<code>name</code>	<code>string</code> , például <code>framework</code>
<code>fullName</code>	<code>string</code> , például <code>laravel/framework</code>
<code>url</code>	<code>string</code> , webes repository-URL
<code>description</code>	<code>?string</code>

A readonly `GitHubRepository` ezt bővíti: `stars: int`, `forks: int`, `language: ?string`, `archived: bool`.

A közös interface `list<RemoteRepository>` típust ígér; a konkrét `GitHub`-metódus pontosabb `list<GitHubRepository>` típust dokumentál. A `GitHub`-mezők így csak megfelelően típusozott `GitHub`-objektumon érhetők el. Natív PHP-típusok, pontos PHPDoc és Larastan ellenőrzés; általános `mixed` metadata-zsák nélkül.

GitHub működés

- Laravel HTTP client és dependency injection; a `GitProvider` alapértelmezett implementációja `GitHubProvider`. `getName()` értéke `GitHub`.
- Fiókfelderítés: `GET /users/{name}`, a válasz `login` és `type` mezőjéből. A `GitHub` névellenőrzése és normalizálása kizárólag ebben az implementációban él.
- User repository-k: `/users/{name}/repos`, `type=owner`. Organization repository-k: `/orgs/{name}/repos`, `type=public`. Csak a source saját publikus repository-jai szerepelnek, forkokkal

és archivált elemekkel együtt.

- Teljes lista: oldalanként legfeljebb 100 elem, `full\_name` szerinti növekvő rendezés, a `Link` fejléc következő oldalának követése. Azonosító szerinti deduplikálás; hibás vagy ismétlődő lapozási link esetén hiba, csonka sikeres eredmény helyett.
- Fix `https://api.github.com` cél, GitHub JSON Accept fejléc, User-Agent és rögzített `2026-03-10` API-verzió. Kapcsolódási timeout 3 másodperc, kérés timeout 10 másodperc.
- Minden kérés hitelesítés nélkül fut. A helyi `.env` felesleges `GITHUB\_PAT` bejegyzése kikerül; tokenkonfiguráció nem készül.
- A külső JSON mezőit ellenőrizzük az objektumképzés előtt. Hibás mezőtípus vagy ismeretlen accounttípus válaszhibát okoz.
- Közös `GitProviderException` hibacsalád, külön felismerhető source-not-found és rate-limit kivételekkel. Hálózati vagy API-hiba nem válhat üres listává. V1-ben nincs automatikus újrapróbálás vagy cache.

Token nélkül az alapteret ****60 kérés/óra, IP-címenként****; minden lapozási kérés fogyasztja. Rate limit esetén a kivétel elérhetővé teszi az újrapróbálás időpontját, ha a GitHub megadja. [GitHub rate limits \(https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api\)](https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api)

Ellenőrzés és elfogadás

Pest tesztek, `Http::fake()` és `Http::preventStrayRequests()` használatával:

- User és organization felismerése; source-getterek, delegálás és provider-egyezés.
- Közös mezők és GitHub-metaadatok helyes leképezése, nullable mezőkkel együtt.
- Üres lista, többoldalas teljes lista, duplikációk és hibás lapozás.
- Kizárólag GET-kérések, Authorization fejléc nélkül; megfelelő endpointok és publikus repository-szűrés.
- Nem létező fiók, rate limit, tiltott hozzáférés, serverhiba, kapcsolatmegszakadás és hibás válasz.
- Egyszerű tesztprovider igazolja, hogy a közös source GitHub-függőség nélkül működik.

Elfogadás: célzott tesztek, Pint, Larastan és teljes meglévő tesztcsomag sikeres. A README rövid használati példát és a token nélküli működés korlátját kapja. Új csomag nem szükséges.

Implementáció csak a terv jóváhagyása után.

Final report – 2026-10-02T09:24:05.603Z

Ris, elkészült a terv szerinti implementáció.

- Közös **GitProvider interface [local file not included]**, readonly adatobjektumok és delegáló `GitSource`.
- **GitHub adapter [local file not included]**: publikus repository-k, teljes lapozás, külön metaadatok, típusos hibakezelés.

- `GITHUB\_PAT` eltávolítva; használati példa a [README-ben](#) [local file not included].

`composer test` sikeres: ****Pint, Larastan, 104 teszt / 394 assertion****. A GitHub-esetek HTTP fake-ekkel ellenőrizve.